Machine Learning Zoomcamp

Module 4 — Evaluation Metrics for Classification

Study notes with equations and charts

Based on the DataTalksClub course notes, `04-evaluation`

Condensed and re-organised from
https://github.com/DataTalksClub/machine-learning-zoomcamp/tree/main/04-evaluation.
All charts are computed on the real Telco churn dataset with the Module 3 model (`random_state=1`).
Numbers can differ from the videos in the 3rd decimal depending on library versions.

Contents

1 What Is a Metric? 2

2 Accuracy and the Dummy Model 2
2.1 The dummy model: accuracy with no model at all . . . 2

3 The Confusion Table 3

4 Precision and Recall 5
4.1 The trade-off and F1 (from "Explore more") . . . 5

5 ROC Curves 6
5.1 TPR and FPR . . . 6
5.2 Reference models . . . 6
5.3 The ROC curve . . . 6

6 ROC AUC 6
6.1 Area under the curve . . . 6
6.2 AUC as a probability . . . 7

7 Cross-Validation 8

8 Summary 9

A Appendix: Choosing the Right Metric 10
A.1 Which one, when? . . . 10
A.2 Why AUC ignores class balance but precision does not . . . 10
A.3 Why AUC is a probability . . . 10

1 What Is a Metric?

A **metric** is a function that takes the predictions and the true values and returns *one number* saying how good the predictions are:

$$\text{metric}: \ (\hat{\mathbf{y}},\ \mathbf{y}) \ \longmapsto \ \mathbb{R}.$$

We keep the Module 3 churn model: logistic regression on 16 categorical + 3 numerical features, trained on 60% of the data. On the 1 409 validation customers it outputs a churn *score* $s_i = g(\mathbf{x}_i) \in [0, 1]$, and a **threshold** t turns scores into decisions:

$$\hat{y}_i(t) = \mathbf{1}[\, s_i \geq t \,].$$

```
y_pred = model.predict_proba(X_val)[:, 1] # scores s_i
churn_decision = (y_pred >= 0.5) # decisions at t = 0.5
```

Every metric in this module except AUC depends on t; AUC looks at all thresholds at once.

2 Accuracy and the Dummy Model

$$\text{accuracy}(t) = \frac{1}{m}\sum_{i=1}^{m} \mathbf{1}\big[\hat{y}_i(t) = y_i\big] = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}.$$

At $t = 0.5$: $1132/1409 = 0.803$ (`accuracy_score(y_val, y_pred >= 0.5)`).

Is 0.5 the best threshold? Scanning $t \in \{0, 0.05, \ldots, 1\}$ shows accuracy peaking at $t = 0.5$ here, but on other problems the best t is often elsewhere, and this loop is how you find it.

2.1 The dummy model: accuracy with no model at all

Before trusting "80% accurate", check what you would get by *not modelling at all*.

1. What it does. The dummy model ignores every feature and gives everyone the same answer: "this customer will not churn".

2. Why $t = 1$ produces it. The rule is "predict churn if $s_i \geq t$". Scores come out of the sigmoid, which never reaches 1, so with $t = 1$ the condition is never true and every customer is predicted as no churn. `Counter(y_pred >= 1.0)` confirms it: {`False: 1409`}, i.e. all 1 409 predictions are "no churn" and none is "churn".

3. Why its accuracy is $1 - \bar{y}$.

- $\bar{y}$ ("y-bar") is the mean of the validation targets, and it equals the churn rate — see the box below for why.
- Saying "no churn" to everyone is right exactly for the customers who stayed: $1409 - 386 = 1023$ of them.

Why the mean of a 0/1 column is the churn rate

The mean is the sum divided by the count: $\bar{y} = \frac{1}{m}\sum_{i=1}^{m} y_i$. Since every y_i is either 0 or 1, the zeros add nothing and each churner adds exactly 1, so the sum simply *counts the churners*:

$$\sum_{i=1}^{m} y_i = \#\{i : \ y_i = 1\} \qquad \Longrightarrow \qquad \bar{y} = \frac{\#\text{churners}}{m} = \text{share of churners} = \text{churn rate}.$$

Tiny example. Five customers with $\mathbf{y} = (0, 1, 0, 0, 1)$: sum = 2 churners, mean = $2/5 = 0.4$, i.e. 40% churned.
Our validation set. $m = 1409$ customers, of whom 386 churned:

$$\bar{y} = \frac{0 + 1 + 0 + \ldots}{1409} = \frac{386}{1409} = 0.274 \quad (27.4\% \text{ churned}, \ 72.6\% \text{ stayed}).$$

```
y_val.sum() # 386 -> number of churners (sum of the 1s)
len(y_val) # 1409 -> number of customers
y_val.mean() # 0.274 -> 386 / 1409 = churn rate
```

This trick works for any binary column: its mean is always the fraction of 1s (e.g. `(y_pred >= 0.5).mean()` is the share of customers we flag).

$$\text{accuracy}_{\text{dummy}} = \frac{\#\text{stayers}}{m} = \frac{1023}{1409} = 1 - \frac{386}{1409} = 1 - \bar{y} = 0.726.$$

In confusion-table terms (§3): TN = 1023, FN = 386, TP = FP = 0. It never catches a churner, so its **recall is 0**.

4. Why it matters.

	accuracy	error rate
dummy ("nobody churns")	0.726	27.4%
logistic regression ($t = 0.5$)	0.803	19.7%

72.6% comes for free, because most customers do not churn anyway. The model's real contribution is the step from 0.726 to 0.803; equivalently, it removes $(27.4 - 19.7)/27.4 \approx 28\%$ of the dummy's errors.

5. General rule. A dummy model's accuracy always equals *the share of the most common class*. With 99% legitimate transactions, a fraud detector that never flags anything scores 99% accuracy and is useless. Always compare against this baseline; scikit-learn has one built in:

```
from sklearn.dummy import DummyClassifier
dummy = DummyClassifier(strategy='most_frequent').fit(X_train, y_train)
dummy.score(X_val, y_val) # 0.726
```

Class imbalance

Only 27% of customers churn. When one class dominates, always predicting the majority class already scores high, so accuracy can look good while the model is useless for the minority class — the very class we care about.

3 The Confusion Table

Every prediction lands in one of four cells. "Positive/negative" is what we *predicted*; "true/false" is whether it was *right*.

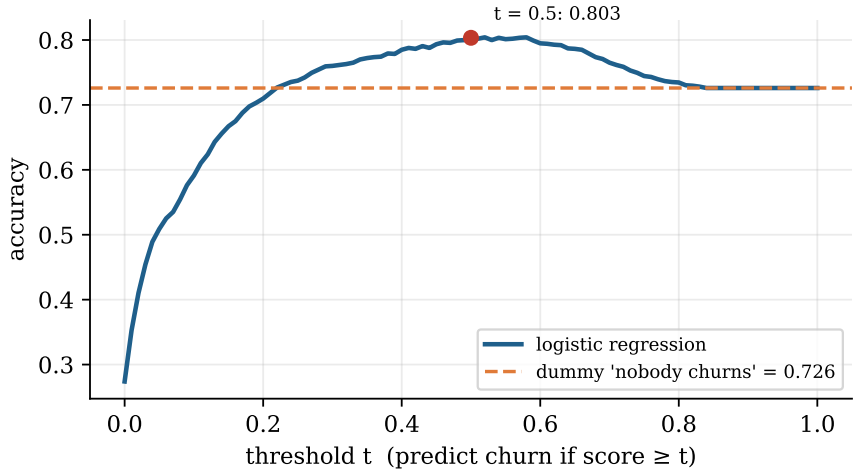


Figure 1: Accuracy vs threshold on validation. The model peaks at 0.803 for $t = 0.5$; at $t \to 1$ it collapses to the dummy model's 0.726. The real gain over doing nothing is 7.7 points, not 80.

		Predicted	
		no churn ($\hat{y} = 0$)	churn ($\hat{y} = 1$)
Actual	no churn ($y = 0$)	TN	FP
	churn ($y = 1$)	FN	TP

- **FP** (false positive): we email a discount to a customer who was never going to leave — wasted money.
- **FN** (false negative): we do nothing and the customer leaves — lost revenue.

```
actual_positive = (y_val == 1); actual_negative = (y_val == 0)
predict_positive = (y_pred >= t); predict_negative = (y_pred < t)
tp = (predict_positive & actual_positive).sum() # & = element-wise AND
tn = (predict_negative & actual_negative).sum()
fp = (predict_positive & actual_negative).sum()
fn = (predict_negative & actual_positive).sum()
```

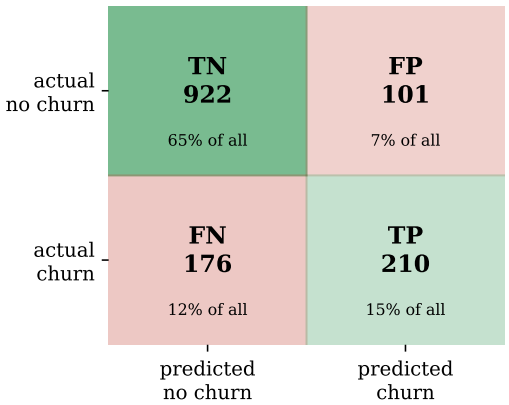


Figure 2: Confusion table at $t = 0.5$: 922 + 210 correct (green), 101 + 176 errors (red); the four cells sum to 1 409.

Accuracy is the diagonal over the total, $(922 + 210)/1409 = 0.803$. The table shows what accuracy hides: the 20% of errors split into 7% false positives and 12% false negatives.

4 Precision and Recall

Both focus on the *positive* class, but each reads a different line of the table:

$$\text{precision} = P = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{recall} = R = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Precision reads the *predicted-churn column*; recall reads the *actual-churn row*:

precision

TN 922	FP 101
FN 176	TP 210

$$\frac{210}{210 + 101} = 0.675$$

recall

TN 922	FP 101
FN 176	TP 210

$$\frac{210}{210 + 176} = 0.544$$

- **Precision 67%**: of the 311 customers we email, a third were never going to leave.
- **Recall 54%**: of the 386 customers who actually left, we missed 176 — almost half.

Accuracy said "80%, close to fine"; precision and recall say the model misses half of the churners. Churners are only 27% of the data, so those misses barely move accuracy.

4.1 The trade-off and F1 (from "Explore more")

Raising t flags fewer customers: those flagged are more certain (precision ↑) but more churners slip through (recall ↓). The **F1 score** combines both with the harmonic mean, which is low whenever either one is low:

$$F_1 = \frac{2PR}{P + R} = \frac{2 \cdot 0.675 \cdot 0.544}{0.675 + 0.544} = 0.603 \text{ at } t = 0.5; \qquad \max_t F_1 = 0.632 \text{ at } t \approx 0.29.$$

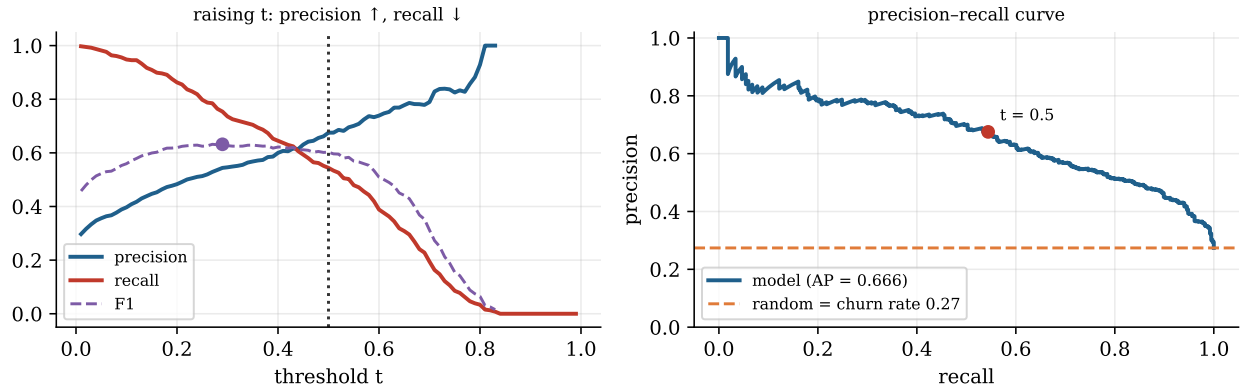


Figure 3: Left: precision, recall and F1 vs threshold; F1 peaks at $t \approx 0.29$, not 0.5. Right: the precision–recall curve (the "Explore more" exercise). A random model sits at precision = churn rate 0.27; the area under the curve (average precision) is 0.666.

5 ROC Curves

5.1 TPR and FPR

ROC (Receiver Operating Characteristic, from WWII radar engineering) uses two rates, one per row of the table:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \ (= \text{recall; maximise}), \qquad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \ (\text{minimise}).$$

At $t = 0.5$: TPR $= 0.544$, FPR $= 101/1023 = 0.099$ — we bother 10% of the loyal customers. Both rates are ratios *within* a true class, so neither depends on how many churners there are.

5.2 Reference models

- **Random model**: uniform scores `np.random.uniform(0, 1, size=len(y_val))`. Accuracy at 0.5 is 0.502. For $s \sim U(0,1)$ the chance of $s \geq t$ is $1 - t$ for *every* customer, so TPR $=$ FPR $= 1 - t$: it cannot tell the classes apart.
- **Ideal model**: put the 1 023 negatives first, then the 386 positives, with scores spread evenly from 0 to 1 (`np.linspace`). Every churner outranks every loyal customer, and the threshold $1023/1409 = 0.726$ separates them perfectly (accuracy 1.0).

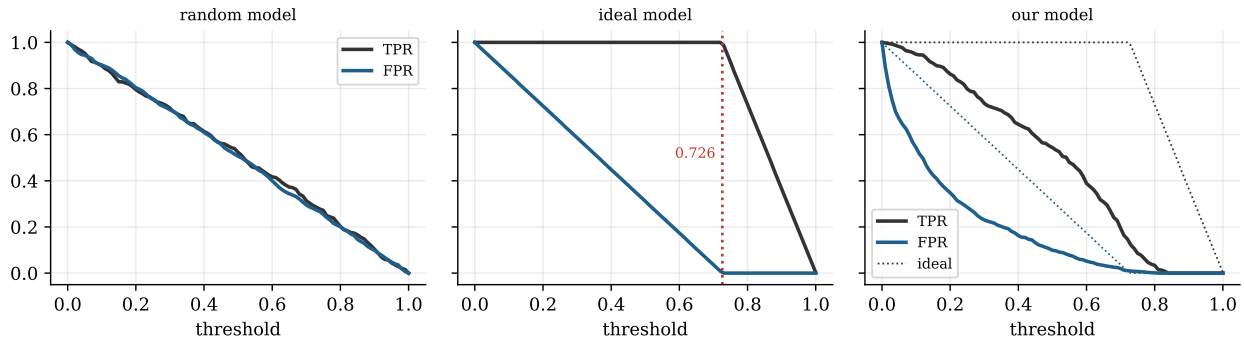


Figure 4: TPR (black) and FPR (blue) vs threshold. Random: two straight lines $1 - t$. Ideal: TPR stays at 1 while FPR drops to 0 at $t = 0.726$. Ours (dotted = ideal): TPR falls too early, FPR too late; the gap is the room to improve.

5.3 The ROC curve

Plot TPR against FPR, one point per threshold. $t = 1$ sits at $(0, 0)$ (flag nobody), $t = 0$ at $(1, 1)$ (flag everybody).

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_val, y_pred)
plt.plot(fpr, tpr); plt.plot([0, 1], [0, 1], linestyle='--') # model vs random
```

6 ROC AUC

6.1 Area under the curve

$$\text{AUC} = \int_0^1 \text{TPR}\, d(\text{FPR}) \in [0, 1], \qquad \text{random} = 0.5, \quad \text{ideal} = 1.$$

Rule of thumb: 0.6 poor, 0.8 good, 0.9 great.

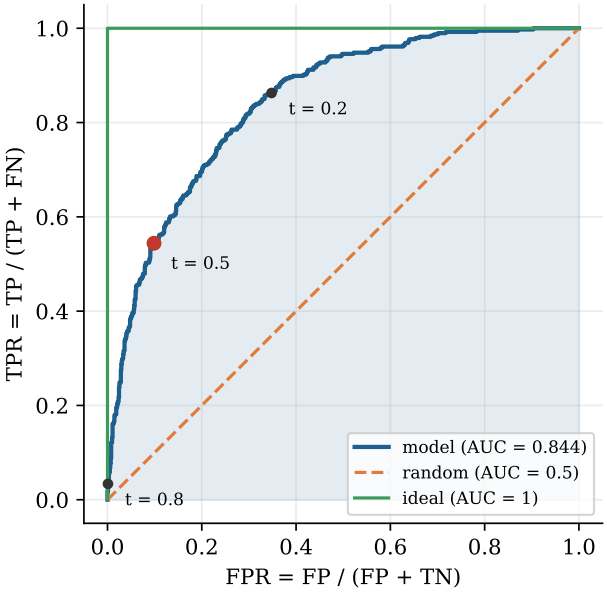


Figure 5: ROC curve of the churn model. Random = diagonal, ideal = the top-left corner. The shaded region is the AUC. Moving along the curve trades more caught churners for more bothered loyal customers.

```
from sklearn.metrics import auc, roc_auc_score
auc(fpr, tpr) # 0.844 (area under ANY curve, trapezoids)
roc_auc_score(y_val, y_pred) # 0.844 (ROC curve + area in one step)
```

Our model: **0.844**. The hand-made 101-threshold table gives 0.8438, the ideal model 0.9999, random scores 0.506.

6.2 AUC as a probability

Interpretation

AUC is the probability that a randomly chosen positive gets a higher score than a randomly chosen negative:

$$\text{AUC} = P\big(s^+ > s^-\big) = \frac{1}{n_+ n_-} \sum_{i:\, y_i=1} \sum_{j:\, y_j=0} \mathbf{1}\big[s_i > s_j\big] \quad \text{(ties count } \tfrac{1}{2}\text{)}.$$

Here: pick a random churner and a random loyal customer — 84% of the time the model ranks the churner as riskier.

```
pos = y_pred[y_val == 1]; neg = y_pred[y_val == 0]
pos_ind = np.random.randint(0, len(pos), size=50000)
neg_ind = np.random.randint(0, len(neg), size=50000)
(pos[pos_ind] > neg[neg_ind]).mean() # 0.846 with seed 1
(pos[:, None] > neg[None, :]).mean() # all 386 x 1023 pairs: 0.8438 = AUC exactly
```

Because AUC is about *ranking*, it does not depend on a threshold or on the class balance — which is why it is so popular for binary classification.

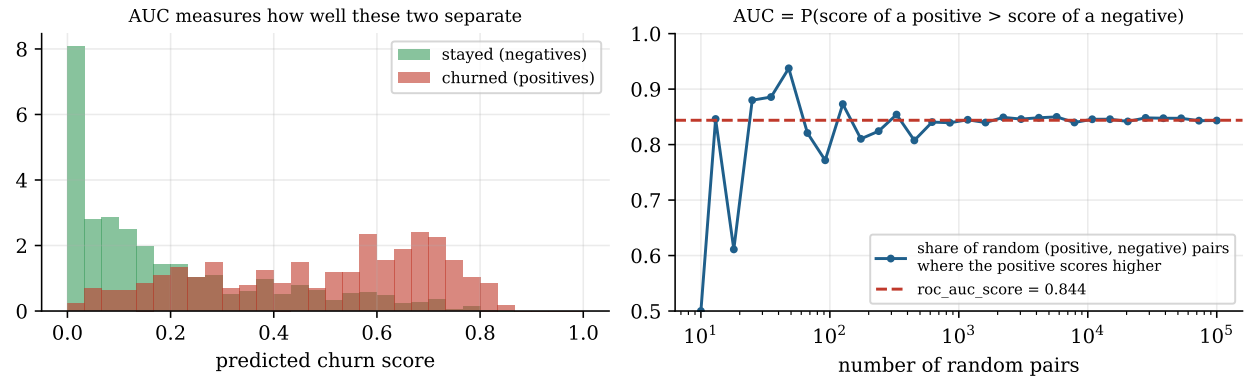


Figure 6: Left: score distributions of churners and stayers; the more they overlap, the lower the AUC. Right: the share of random pairs where the churner scores higher converges to the AUC as the number of pairs grows.

7 Cross-Validation

One validation split gives one number, which might be lucky. ***k*-fold cross-validation** splits the full training set into k folds, trains on $k-1$ and validates on the remaining one, k times:

iteration 1	val	train	train	train	train	$\rightarrow \mathrm{AUC}_1$
iteration 2	train	val	train	train	train	$\rightarrow \mathrm{AUC}_2$
iteration 3	train	train	val	train	train	$\rightarrow \mathrm{AUC}_3 \Rightarrow \overline{\mathrm{AUC}} \pm \sigma$
iteration 4	train	train	train	val	train	$\rightarrow \mathrm{AUC}_4$
iteration 5	train	train	train	train	val	$\rightarrow \mathrm{AUC}_5$

$$\overline{\mathrm{AUC}} = \frac{1}{k}\sum_{f=1}^{k} \mathrm{AUC}_f, \qquad \sigma = \sqrt{\frac{1}{k}\sum_{f=1}^{k}\left(\mathrm{AUC}_f - \overline{\mathrm{AUC}}\right)^2}.$$

The mean estimates the performance; the standard deviation says how stable the model is across splits.

```
def train(df_train, y_train, C=1.0):
    dicts = df_train[categorical + numerical].to_dict(orient='records')
    dv = DictVectorizer(sparse=False); X_train = dv.fit_transform(dicts)
    model = LogisticRegression(C=C, max_iter=1000).fit(X_train, y_train)
    return dv, model

def predict(df, dv, model):
    X = dv.transform(df[categorical + numerical].to_dict(orient='records'))
    return model.predict_proba(X)[:, 1]

for C in [0.001, 0.01, 0.1, 0.5, 1, 5, 10]:
    kfold = KFold(n_splits=5, shuffle=True, random_state=1)
    scores = []
    for train_idx, val_idx in kfold.split(df_full_train): # a generator
        df_tr, df_va = df_full_train.iloc[train_idx], df_full_train.iloc[val_idx]
        dv, model = train(df_tr, df_tr.churn.values, C=C)
        scores.append(roc_auc_score(df_va.churn.values, predict(df_va, dv, model)))
    print('C=%s␣%.3f␣+-␣%.3f' % (C, np.mean(scores), np.std(scores)))
```

Final model: retrain on all of `df_full_train` with $C = 1$ and evaluate once on test: AUC $= 0.858$, slightly above the CV mean, a normal fluctuation.

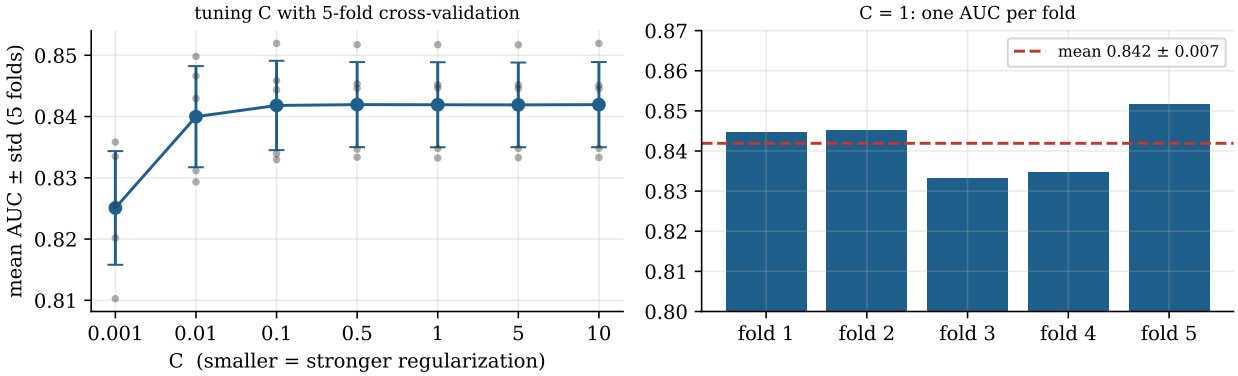


Figure 7: Left: mean ± std AUC over 5 folds for each C (dots = individual folds). Only $C = 0.001$ (heavy regularization) is clearly worse; from 0.01 on the curve is flat at ≈ 0.842, so the default $C = 1$ is fine. Right: the five fold scores for $C = 1$ range from 0.833 to 0.852.

When to use it: large dataset ⇒ one hold-out split is enough and faster; small dataset, or you need to know the spread ⇒ cross-validation. More folds give a better estimate but cost more training runs.

8 Summary

Metric	Meaning	Churn model (val)
Accuracy	$(\text{TP} + \text{TN})/m$; misleading with class imbalance	0.803 (dummy 0.726)
Precision	$\text{TP}/(\text{TP} + \text{FP})$: how many alerts are right	0.675
Recall / TPR	$\text{TP}/(\text{TP} + \text{FN})$: how many churners we catch	0.544
FPR	$\text{FP}/(\text{FP} + \text{TN})$: how many loyal customers we bother	0.099
F1	$2PR/(P + R)$: one number for both	0.603 (best 0.632)
ROC AUC	area under TPR vs FPR $= P(s^+ > s^-)$; threshold-free	0.844 (test 0.858)
k-fold CV	mean ± std over k splits; tunes C	0.842 ± 0.007

Next, Module 5 deploys this churn model as a web service.

A Appendix: Choosing the Right Metric

A.1 Which one, when?

Situation	Look at
Balanced classes, errors cost the same	accuracy
False alarms are expensive (spam filter, costly retention offers)	precision
Misses are expensive (disease screening, fraud)	recall
Need one number at a fixed threshold	F1, or a cost-weighted metric
Ranking quality, threshold not decided yet	ROC AUC
Rare positive class, care only about positives	precision–recall curve / average precision

A.2 Why AUC ignores class balance but precision does not

TPR uses only actual positives and FPR only actual negatives, so duplicating every loyal customer ten times leaves both unchanged and the ROC curve stays the same. Precision mixes the two classes:

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} = \frac{\mathrm{TPR} \cdot n_+}{\mathrm{TPR} \cdot n_+ + \mathrm{FPR} \cdot n_-},$$

so with the same TPR and FPR, more negatives ($n_- \uparrow$) means lower precision. That is why the random baseline of the PR curve is the positive rate (0.27 here), while for ROC it is always the diagonal.

A.3 Why AUC is a probability

Sort all customers by score. Moving the threshold down past a *positive* moves the ROC curve up by $1/n_+$; past a *negative*, right by $1/n_-$. Each negative's step adds a strip of area of width $1/n_-$ and height equal to the share of positives already above it, i.e. the share of positives that outscore that negative. Summing over all negatives:

$$\mathrm{AUC} = \sum_{j:\, y_j = 0} \frac{1}{n_-} \cdot \frac{\#\{i :\, y_i = 1,\ s_i > s_j\}}{n_+} = P(s^+ > s^-),$$

the normalised Mann–Whitney U statistic from classical statistics. Computing all 386×1023 pairs reproduces `roc_auc_score` exactly (0.84385).